Part 1

1. My proposed Row Key for this dataset is City#Make#DOLvehicleID. I put City first because cities have a more even distribution than makes. With makes, there are only 52 unique values, and Tesla is a clear outlier with a much larger share of the data than any other make. However, the dataset has 934 distinct cities, and no single city dominates the data the way Tesla does. If Make came first, then all of the Tesla data records would be stored next to each other and the data would be grouped unevenly, which slows performance. This is why I choose to put the City first.

   I used the DOL Vehicle ID instead of the VIN because VIN only takes the first 10 characters of an ID, so it is not unique to each vehicle and data record. This guarantees that no vehicles are accidentally merged together or overwrite one another.

   This key avoids hotspotting because it starts with City, so transactions (both reads and writes) are spread more evenly across the involved machines.

   This impacts my ability to efficiently query vehicles by City or Make because City-only queries become fast (all records for a city sit next to each other and can be read quickly), City and Make queries are even faster (search is narrowed even more when you specify the make), but Make-only queries are slower (each make's vehicles could be across any city and the whole table needs to be scanned).

2. I would use 3 column families. The 'identity' family would have VIN (1-10) and DOL Vehicle ID. The 'vehicle' family would have Model Year, Make, Model, Electric Vehicle Type, Clean Alternative Fuel Vehicle (CAFV) Eligibility, and Electric Range. The 'location' family would have County, City, State, Postal Code, Legislative District, Vehicle Location, Electric Utility, and 2020 GEOID.

   This grouping aids in distributed read performance because Bigtable stores column families separately, so each query only looks at the families it is involved with, and it completely skips irrelevant data. Since less data is read overall, this results in lower latency for reads.

3. The pro of using a Hashed Shard Key is that it spreads data evenly across machines, so no machine has a significantly greater workload than the others. Since the data is spread evenly, we can also avoid hotspots when reading and writing, because new records are distributed across the cluster instead of going to a single machine, and lookups are spread out instead of all relying on one machine.

   The pro of using a Range-Based Shard Key is that it is easy to scan or query over a range of years, since those values are stored together on the same machine(s). Any irrelevant machine can be skipped entirely. It is also easy to remove older values that should no longer be considered.

The con of using a Hashed Shard Key is that you cannot look at neighboring values together without searching through every machine, since those values could be scattered anywhere across the machines.

The con of using a Range-Based Shard Key is that the load on machines is uneven, so some may have to store more data as well as handle a heavier workload. Hotspots are also prone to occur for both reads and writes, since new vehicle registrations land on the single machine handling the current year, and they are not spread out.

Part 4

The average latency for /insert-fast is 33.3492 ms.
The average latency for /insert-safe is 35.7649 ms.

Based on these numbers and the CAP theorem, I would choose /insert-fast (faster primary-only option) if I were counting likes or views on a social media platform. Social media apps usually have to handle a large number of writes/updates/inserts that occur very frequently (multiple within each second), so the time savings of /insert-fast will add up over time. Missing a few likes or views is low-stakes, because the count would only be off by a small amount. In the CAP theorem, this means preferring availability over consistency, because speed matters more than double-checking everything.

